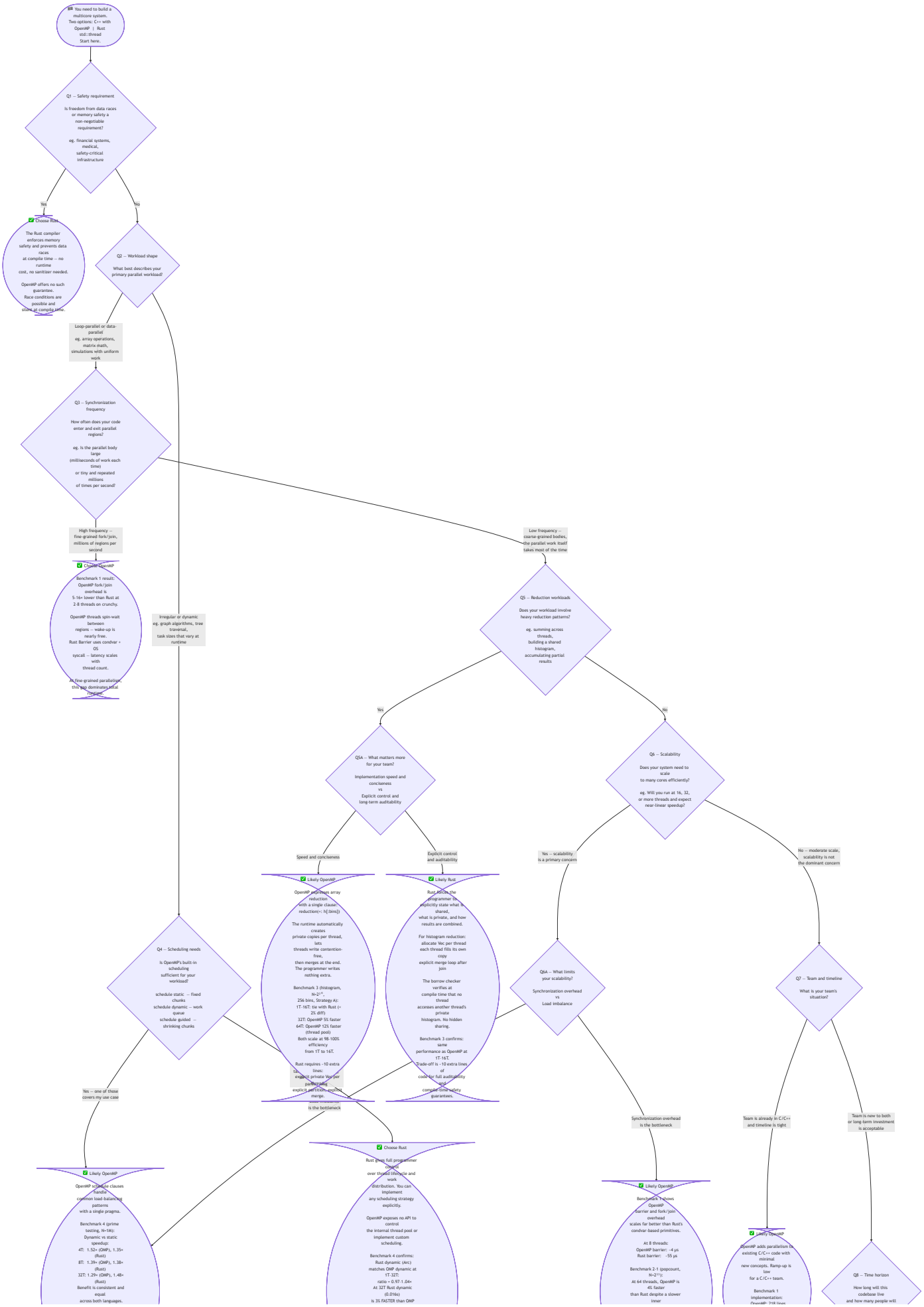
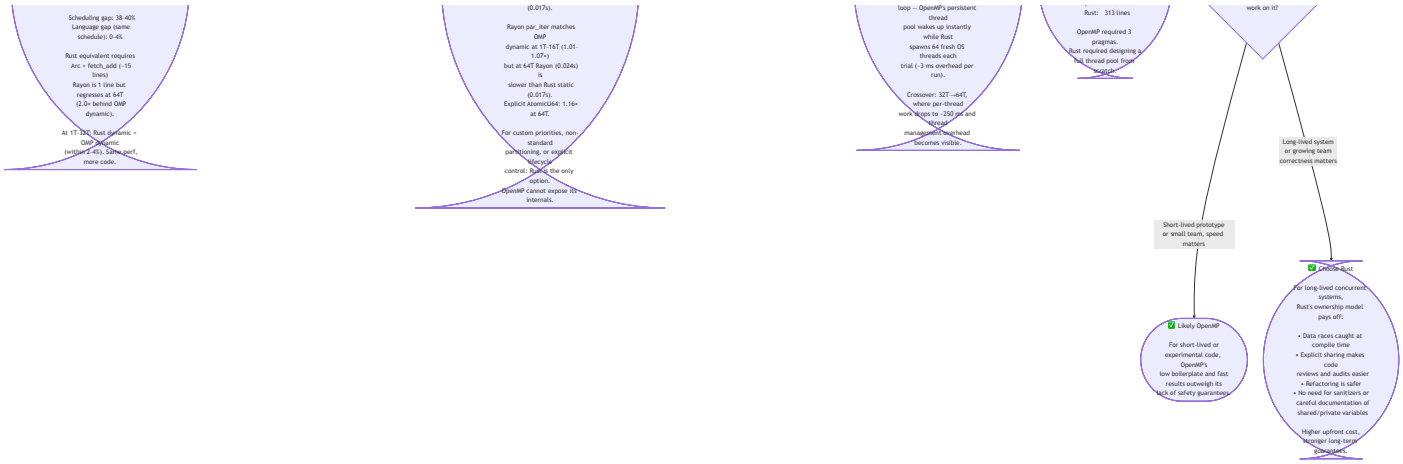
Decision Flowchart: C++/OpenMP vs Rust for Multicore Systems

> **How to use this flowchart:** Walk through each question from top to bottom. Answer based on your system's requirements. Each terminal outcome cites which benchmark provides the supporting evidence. Nodes marked **[TBD]** will be updated as Benchmarks 2–4 are completed.

Flowchart


You need to build a multicore system. Two options: C++ with OpenMP | Rust std::thread Start here.
Q1 — Safety requirement
Is freedom from data races or memory safety a non-negotiable requirement?
eg. financial systems, medical, safety-critical infrastructure
Yes
No
✅ Choose Rust
The Rust compiler enforces memory safety and prevents data races at compile time — no runtime cost, no sanitizer needed.
OpenMP offers no such guarantee. Race conditions are possible and silent at compile time.
Q2 — Workload shape
What best describes your primary parallel workload?
Loop-parallel or data-parallel eg. array operations, matrix math, simulations with uniform work
Q3 — Synchronization frequency
How often does your code enter and exit parallel regions?
eg. Is the parallel body large (milliseconds of work each time) or tiny and repeated millions of times per second?
High frequency — fine-grained fork/join, millions of regions per second
✅ Choose OpenMP
Benchmark 1 result: OpenMP fork/join overhead is 5-16× lower than Rust at 2-8 threads on crunchy.
OpenMP threads spin-wait between regions — wake-up is nearly free. Rust Barrier uses condvar + OS syscall — latency scales with thread count.
At fine-grained parallelism, this gap dominates total runtime.
Irregular or dynamic eg. graph algorithms, tree traversal, task sizes that vary at runtime
Low frequency — coarse-grained bodies, the parallel work itself takes most of the time
Q5 — Reduction workloads
Does your workload involve heavy reduction patterns?
eg. summing across threads, building a shared histogram, accumulating partial results
Yes
No
Q5A — What matters more for your team?
Implementation speed and conciseness vs Explicit control and long-term auditability
Speed and conciseness
Explicit control and auditability
✅ Likely OpenMP
OpenMP expresses array reduction with a single clause: reduction(+: h[:bins])
The runtime automatically creates private copies per thread, lets threads write contention-free, then merges at the end. The programmer writes nothing extra.
Benchmark 3 (histogram, N=2^27, 256 bins, Strategy A): 1T-16T: tie with Rust (< 2% diff) 32T: OpenMP 5% faster 64T: OpenMP 12% faster (thread pool) Both scale at 98-100% efficiency from 1T to 16T.
Rust requires ~10 extra lines: explicit private Vec per thread, explicit merge.
✅ Likely Rust
Rust forces the programmer to explicitly state what is shared, what is private, and how results are combined.
For histogram reduction: allocate Vec per thread each thread fills its own copy explicit merge loop after join
The borrow checker verifies at compile time that no thread accesses another thread's private histogram. No hidden sharing.
Benchmark 3 confirms: same performance as OpenMP at 1T-16T.
Trade-off is ~10 extra lines of code for full auditability and compile-time safety guarantees.
Q6 — Scalability
Does your system need to scale to many cores efficiently?
eg. Will you run at 16, 32, or more threads and expect near-linear speedup?
Yes — scalability is a primary concern
No — moderate scale, scalability is not the dominant concern
Q4 — Scheduling needs
Is OpenMP's built-in scheduling sufficient for your workload?
schedule static — fixed chunks schedule dynamic — work queue schedule guided — shrinking chunks
Yes — one of those covers my use case
Scheduling is the bottleneck
Q6A — What limits your scalability?
Synchronization overhead vs Load imbalance
Synchronization overhead is the bottleneck
Q7 — Team and timeline
What is your team's situation?
Team is already in C/C++ and timeline is tight
Team is new to both or long-term investment is acceptable
✅ Likely OpenMP
OpenMP schedule clauses handle common load-balancing patterns with a single pragma.
Benchmark 4 (prime testing, N=1M): Dynamic vs static speedup: 4T: 1.52× (OMP), 1.35× (Rust) 8T: 1.39× (OMP), 1.38× (Rust) 32T: 1.29× (OMP), 1.48× (Rust) Benefit is consistent and equal across both languages.
✅ Choose Rust
Rust gives full programmer control over thread lifecycle and work distribution. You can implement any scheduling strategy explicitly.
OpenMP exposes no API to control the internal thread pool or implement custom scheduling.
Benchmark 4 confirms: Rust dynamic (4rc) matches OMP dynamic at 1T-32T: ratio = 0.97-1.04× At 32T Rust dynamic (0.016s) is 3% FASTER than OMP
✅ Likely OpenMP
Benchmark 5 shows OpenMP barrier and fork/join overhead scales far better than Rust's condvar-based primitives.
At 8 threads: OpenMP barrier: ~4 µs Rust barrier: ~55 µs
Benchmark 2-1 (popcount, N=2^33): At 64 threads, OpenMP is 4% faster than Rust despite a slower inner
✅ Likely OpenMP
OpenMP adds parallelism to existing C/C++ code with minimal new concepts. Ramp-up is low for a C/C++ team.
Benchmark 1 implementation: OpenMP: 218 lines
Q8 — Time horizon
How long will this codebase live and how many people will



Evidence Map

Each decision node in the flowchart is informed by one or more benchmarks. This table will be filled in as benchmarks are completed.

Question	Key metric	Benchmark	Status
Q3 — Sync frequency	Fork/join overhead vs thread count	Benchmark 1	✅ Complete
Q5 — Reduction workloads	Reduction LOC, runtime, correctness effort	Benchmark 3	✅ Complete
Q6A — Sync overhead at scale	Barrier cost, speedup curves	Benchmark 1 + 2-1	✅ Complete
Q6A — Load imbalance	Runtime under uneven work, scheduling	Benchmark 4	✅ Complete
Q4 — Custom scheduling	LOC, flexibility, runtime comparison	Benchmark 4	✅ Complete
Q6 — Scalability	Speedup and efficiency vs thread count	Benchmark 2-1	✅ Complete

Key Findings (Benchmark 1 — Thread Overhead)

These facts are established from the thread overhead microbenchmark (fork/join, barrier, atomic, 2–8 threads):

1. **OpenMP fork/join overhead is 5–16× lower than Rust** at 2–8 threads. → Drives Q3: if sync frequency is high, OpenMP wins.
2. **Barrier overhead is equal at 1 thread, then diverges sharply.** OpenMP: ~2–4 µs. Rust: ~16–55 µs at 2–8 threads. → Reinforces Q6A: if synchronization is the bottleneck, OpenMP scales better.

3. **Atomic increment cost is identical** (~24–102 ns on both sides, within noise). → Neither language has an advantage on hardware atomics.

4. **OpenMP provides no programmer control over thread lifecycle.** You cannot force thread creation or destruction between regions. → Drives Q4: if custom scheduling or lifecycle control is needed, OpenMP cannot do it.

5. **Rust required 95 more lines of code** and a full thread pool design for an equivalent benchmark. → Drives Q7/Q8: for rapid development, OpenMP wins on productivity.

6. **Rust's borrow checker caught all sharing bugs at compile time.** No runtime debugging needed. → Drives Q1/Q8: for safety-critical or long-lived systems, Rust's upfront cost is justified.

Key Findings (Benchmark 2 — Monte Carlo Pi)

These facts are established from the Monte Carlo Pi benchmark (Xorshift64 RNG, 1B samples, 1–64 threads):

7. **RNG choice introduced a 3× artificial performance gap.** The original comparison used mt19937_64 (C++ stdlib) vs Xorshift64 (Rust). Unifying both to Xorshift64 collapsed the gap to ~1.35×. Confounding factors in the inner loop must be eliminated before comparing parallel models. → Lesson: benchmark design determines what you are actually measuring. A 3× gap from RNG choice was masking the true parallelism comparison.

8. **With unified RNG, OpenMP is ~1.35× faster at all thread counts — a constant multiplier.** A constant ratio across all thread counts means the difference is entirely in single-thread code generation (Xorshift64's sequential dependency chain preventing SIMD), not in parallelism quality. Both models scale at the same rate. → Precursor to Benchmark 2-1: the FP and RNG inner loop must be replaced with a bias-free workload to isolate the parallelism model.

Key Findings (Benchmark 2-1 — Embarrassingly Parallel Scalability, Popcount)

These facts are established from the popcount benchmark ($N = 2^{33}$, 1–64 threads, both implementations clean):

9. **Both OpenMP and Rust scale at near-identical efficiency for compute-bound embarrassingly parallel work.** Both reach 97–100% parallel efficiency from 1T to 16T. Neither parallelism model has a structural scalability advantage when synchronization is absent. → Q6 confirmed: if scalability is the concern, the decision must be made on other axes.

10. **Rust/LLVM is ~1.13–1.15× faster than C++/GCC from 1T to 16T.** The gap is a **constant multiplier** — it does not change as thread count scales. This proves the difference is in single-thread code generation (LLVM's 8× loop unrolling vs GCC's scalar loop), not in the parallelism model. → Reinforces Q7/Q8: LLVM produces better code for this workload, but this does not affect the parallel scaling decision.

11. **At 64 threads, OpenMP reverses the advantage and runs 4% faster than Rust.** With 64 threads each doing only ~~250ms of compute, Rust's cost of spawning 64 fresh OS threads per trial (~~3ms total) erases its inner-loop speed advantage. OpenMP's persistent thread pool wakes up at nearly zero cost. → Reinforces Q3: fine-grained or high-thread-count parallelism favors OpenMP's thread pool model.

12. **Neither compiler uses AVX2 SIMD for popcount on crunchy.** AVX-512 VPOPCNTDQ is not available on the cluster. Both binaries use scalar `popcnt`. LLVM's advantage comes from 8× loop unrolling and multiple

accumulators, not SIMD width.

Key Findings (Benchmark 2-2 — Rayon vs std::thread)

These facts are established from repeating the popcount benchmark with Rayon (N = 2^{33}, 1–64 threads):

13. **Switching from bare `std::thread` to Rayon does not fix the 64T reversal — it makes it larger.** OpenMP leads Rayon by 7.5% at 64T vs only 4% over std::thread. Rayon's work-stealing bookkeeping across 64 workers compounds the thread management overhead. → The 64T reversal is not caused by thread spawn cost alone; it is a structural property of OpenMP's persistent pool vs any fresh-thread or work-stealing model.

14. **For uniform workloads, bare `std::thread` with static partitioning beats Rayon.** std::thread is 3–5% faster than Rayon at every thread count from 1T to 32T. LLVM generates an 8× unrolled loop inside the bare thread closure vs only 4× inside Rayon's work-stealing closure. Work-stealing adds scheduler overhead that buys nothing when every element costs the same. → Rayon's advantage is for **irregular** workloads where load imbalance exists. For perfectly uniform work, static partitioning is always better.

15. **The performance ranking for uniform workloads is: std::thread > Rayon > OpenMP (1T–32T); OpenMP > std::thread > Rayon (64T).** All three scale at equivalent efficiency. The differences are in single-thread code quality and thread management overhead, not in parallelism model quality.

Key Findings (Benchmark 3 — Parallel Histogram, Reduction Workloads)

These facts are established from the histogram benchmark (N = 2^{26}, 256 bins, Strategy A, 1–64 threads):

16. **Algorithm design matters more than language choice.** Strategy B (shared atomics) made performance 4× *worse* at 64 threads. Strategy A (private histograms + merge) gives 35× speedup at 64T. Choosing the right reduction pattern is the first decision — language comes second.

17. **Performance is a tie at 1T–16T.** OpenMP and Rust differ by less than 2% at every thread count from 1T to 16T. Both scale at 98–100% parallel efficiency. The Q5A decision is entirely about code, not runtime. → Q5A confirmed: the choice between OpenMP and Rust for reduction workloads is a programmability decision, not a performance decision.

18. **OpenMP's `reduction` clause expresses the entire pattern in one line.** The programmer writes a serial-looking loop; the runtime handles private allocation, contention-free writes, and merge invisibly. Rust requires ~10 explicit lines for the same behavior. → Drives `OPENMP_REDUCE`: for teams that prioritize conciseness and development speed, OpenMP wins on this axis.

19. **Rust's explicit model gives compile-time auditability.** Every decision is visible: what each thread owns, what it writes, and when the merge happens. The borrow checker verifies at compile time that no thread accesses another's private data. → Drives `RUST_REDUCE`: for long-lived systems where correctness and code review clarity matter, Rust's verbosity is a feature.

20. **The 32T→64T cliff is a memory bandwidth wall, not a parallelism problem.** Both languages collapse from ~94% efficiency at 32T to ~50% at 64T. The 256MB input array saturates memory bandwidth at ~32 threads. This is a hardware constraint that neither language can overcome.

Key Findings (Benchmark 4 — Irregular Workload, Prime Testing)

These facts are established from the prime testing benchmark (N = 1,000,000, 5 strategies, 1–64 threads):

21. **Static scheduling has identical load-imbalance penalty in both languages.** With clean data (early-morning low-load run), Rust static and OMP static track each other within ±11% at every thread count: 8T: OMP static = 0.077s, Rust static = 0.077s (tied exactly) 64T: OMP static = 0.018s, Rust static = 0.017s (Rust 6% faster) Both achieve 43% parallel efficiency at 64T. The load imbalance bottleneck is a property of the algorithm, not the language. → Confirms Q6A: for irregular workloads, static scheduling is the wrong choice regardless of language.

22. **Dynamic scheduling delivers 1.3–1.5× speedup over static, identically in both languages.** OMP dynamic/static: 1.52× at 4T, 1.39× at 8T, 1.29× at 32T. Rust dynamic/static: 1.35× at 4T, 1.38× at 8T, 1.48× at 32T. The benefit is consistent and nearly equal between languages. Scheduling mechanism (pragma vs. AtomicU64) does not change how much imbalance is corrected — only how much code is needed. → Confirms Q4 (OPENMP_SCHED): OpenMP's one-keyword change delivers the same benefit as ~15 lines of explicit Rust code.

23. **Rust dynamic matches OMP dynamic within 2–4% from 1T to 32T.** Rust dynamic: 0.446s, 0.220s, 0.110s, 0.056s, 0.029s, 0.016s at 1T/2T/4T/8T/16T/32T. OMP dynamic: 0.432s, 0.217s, 0.109s, 0.055s, 0.028s, 0.017s at same counts. At 32T, Rust dynamic (0.016s) is 3% *faster* than OMP dynamic (0.017s). The explicit `Arc<AtomicU64>` counter achieves the same load-balancing effect as OpenMP's internal work queue. → Confirms Q4 (RUST_SCHED): Rust can match OMP dynamic, but requires ~15 lines vs. one keyword.

24. **Rayon matches OMP dynamic at 1T–16T (1.01–1.07×), then regresses sharply.** Rayon: 0.436s, 0.219s, 0.111s, 0.057s, 0.030s at 1T–16T — within 7% of OMP dynamic. At 32T: Rayon = 0.020s vs OMP dynamic 0.017s → 1.18× behind. At 64T: Rayon = 0.024s vs OMP dynamic 0.012s → **2.0× behind**. At 64T, Rayon (0.024s) is also **41% slower than Rust static (0.017s)** — work-stealing overhead on this 8-NUMA-node machine exceeds any load-balancing benefit at N=1M, 64T. → Nuance for Q4: Rayon is the highest-productivity option at 1T–16T (1 line, near-optimal). Explicit dynamic (AtomicU64) is better at 32T–64T on NUMA hardware.

25. **Scheduling gap (38–40%) vastly exceeds language gap (0–4%).** At 8T (fully clean, all 5 strategies populated): OMP dynamic: 0.055s (best) Rust dynamic: 0.056s (1.02× — language gap) Rust rayon: 0.057s (1.04× — language gap) OMP static: 0.077s (1.40× — scheduling gap) Rust static: 0.077s (1.40× — scheduling gap, same as OMP!) The scheduling gap (40%) is ~10× the language gap (2–4%). For irregular workloads: choose the schedule first, the language second. → Reinforces Q4 over Q7/Q8.

26. **Previously anomalous Rust static 64T results were cluster load, not NUMA.** Earlier runs showed Rust static 64T = 0.165–0.304s (all 5 trials, 10–17× slower than expected). Early-morning low-load run 2 shows Rust static 64T = **0.017s** (five trials: 0.017, 0.017, 0.017, 0.017, 0.018). This definitively rules out NUMA effects or structural load imbalance as the cause — both are reproducible properties that would persist even at low cluster load. The anomaly was pure cluster interference. OpenMP's persistent pool produced 0 contaminated cells in all runs. Rust's fresh-thread spawning is more sensitive to cluster load, but produces identical performance when the cluster is quiet. → Reinforces Q3: OpenMP's persistent pool is a reliability advantage on shared hardware, not just a latency advantage.

How to Read the Final Outcome

The flowchart will rarely give a perfect one-sided answer. In practice:

- If multiple paths point to **OpenMP** → OpenMP is the right fit for your system.
- If multiple paths point to **Rust** → Rust is the right fit.

- If paths are mixed → read the cited benchmark numbers for your specific bottleneck and weight the decision toward whichever axis matters most for your system.

The flowchart is a structured way to surface the right trade-off questions, not a mechanical oracle.